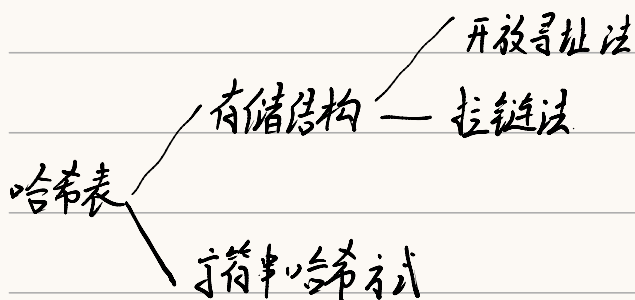


哈希表



哈希表：将一个庞大的空间映射到一个较小的空间 $0 \sim N$ $10^5, 10^6$

应用场景：将一组数量较少但值域很大的数插入到 $0 \sim N$ 的数组中

类似离散化的操作，但不要求有序 离散化是一种特殊的哈希表

$$x \in (-10^9 \sim 10^9), h(x) \in (0, 10^5)$$

$h(x)$ ：哈希函数

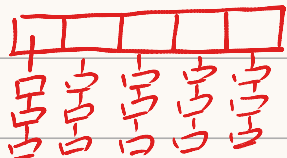
负数、正数 $\rightarrow$ 正数
 $\uparrow$

常见写法：① 直接取表的模 $h(x) = x \bmod 10^5$ $(x \% N + N) \% N$

②：①的方法发生冲突的概率很大（多个数 $h(x)$ 后为同一）

因此可以 $\%$ （不超过表长的最大质数）

冲突处理方式：

① 拉链法： 开一维数组存储所有哈希值

每一格也是一个链表的开头，可视为头指针

插入： $h(x)$ 后，在 $h(x)$ 对应的哈希值链表上头插 x

查询： $h(x)$ 后，遍历对应的链表查询 x

删除：哈希表中一般无真正的删除，我们一般开另外一个数组，对删除的元素做一个标记，代表已删除，而不是真正删除

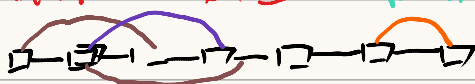
存储方式

$h[N]$ $e[N]$ $ne[N]$

$h[N]$: $h[i]$ 代表哈希值为 i 的链表的头指针

$e[N]$: 多条链表交错组成的一维数组 没有选取 $\begin{matrix} 1 \rightarrow 2 \rightarrow 3 \\ 2 \rightarrow 3 \rightarrow 1 \\ 3 \rightarrow 1 \rightarrow 2 \end{matrix}$ 的二维结构

$ne[N]$ next 指针



② 开放寻址法:

最好为质数, 减少冲突

只开了一个一维数组 (数组长度一般为题目数据 $2 \sim 3$ 倍)

冲突处理: 若 $h[x]$ 位置上有数据了, 就去 $h[x+1]$

$h[x+2] \dots h[x+n]$ 直到找到空位

$(h[x+k] \% \text{Size}) \rightarrow$ 到末尾后再从头找

优化: $h[x+i]$ $i=1, 2, 3 \dots$ 容易找堆

$h[x+i]$ $i=1^2, -1^2, 2^2, -2^2$

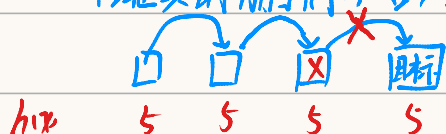
$\dots q^2, -q^2$ ($q \leq \frac{n}{2}$)

添加: $h[x+i]$, $i=0, 1, 2 \dots$ 直到有空位, 插入

查找: $h[x+i]$ $i=0, 1, 2 \dots$ 直到找到 x 或
出现空位 (这代表不存在)

删除: 做个标记, 代表删除

不能真的删除, 否则 查找链会断, 导致误判



$h[N]$ 初始化, 每个元素初始化为数据范围之外的数
以此代表这个位置为空

840. 模拟散列表

题目 提交记录 讨论 题解 视频讲解

维护一个集合，支持如下几种操作：

1. $I\ x$ ，插入一个整数 x ；
2. $Q\ x$ ，询问整数 x 是否在集合中出现过；

现在要进行 N 次操作，对于每个询问操作输出对应的结果。

输入格式

第一行包含整数 N ，表示操作数量。

接下来 N 行，每行包含一个操作指令，操作指令为 $I\ x$ ， $Q\ x$ 中的一种。

输出格式

对于每个询问指令 $Q\ x$ ，输出一个询问结果，如果 x 在集合中出现过，则输出 **Yes**，否则输出 **No**。

每个结果占一行。

数据范围

$1 \leq N \leq 10^5$
 $-10^9 \leq x \leq 10^9$

输入样例：

```
5
I 1
I 2
I 3
Q 2
Q 5
```

输出样例：

```
Yes
No
```

难度：

简单

时空限制：

1s / 64MB

总通过数：

129488

总尝试数：

198545

来源：

模板题

算法标签 -

```
1 // 拉链法
2 #include <stdio.h>
3 #include <string.h>
4
5 const int N = 1e5 + 3; // 最好直接取成质数
6 int n;
7 int h[N], e[N], ne[N], idx;
8 // h[i] 存储哈希值为 i 的对应链表的头节点
9 // e[i] 存储每个点的实际值（插入查找的 x 的实际值）
10 // ne[i] 存储每个结点的下一个结点
11 // idx 用来给每个结点分配唯一的下标
12 // 注意，这里 h 中不存值，只存头指针
13
14 void insert(int x) {
15     int hs = (x % N + N) % N; // 获取哈希值
16     e[idx] = x; ne[idx] = h[hs]; h[hs] = idx;
17     // 存结点的实际值、头插法插入到 h[hs] 对应的链表中
18     idx++;
19     return;
20 }
21
22 int find(int x) {
23     int hs = (x % N + N) % N;
24     // 遍历当前哈希值对应的链表
25     for(int i = h[hs]; i != -1; i = ne[i]) {
26         if(e[i] == x) return 1;
27     }
28     return 0;
29 }
30
31
32 int main() {
33     scanf("%d", &n);
34     // 初始化，一开始所有哈希值链表头指针都指向空 (-1)
35     memset(h, -1, sizeof(h));
36
37     char choose[5]; int x;
38     for(int i = 0; i < n; i++) {
39         scanf("%s", choose);
40         if(choose[0] == 'I') {
41             scanf("%d", &x);
42             insert(x);
43         } else {
44             scanf("%d", &x);
45             if(find(x)) printf("Yes\n");
46             else printf("No\n");
47         }
48     }
49
50     return 0;
51 }
52
```

```
56 // 开放寻址法
57 #include <stdio.h>
58 #include <string.h>
59
60 const int N = 2e5 + 3, null = 0x3f3f3f3f;
61 int n;
62 int h[N];
63
64 void insert(int x) {
65     int hs = (x % N + N) % N;
66     while(h[hs] != null) {
67         hs++;
68         if(hs == N) hs = 0;
69     };
70     // 也可写成 hs = (hs + 1) % N, 但是影响效率
71     // 这里 % N, 当 hs 达到模长时会自动回 0
72     h[hs] = x;
73     return;
74 }
75
76 int find(int x) {
77     int hs = (x % N + N) % N;
78     while(1) {
79         // 因为 N 开的是二倍的数据量，所以这个循环一定会停下来的
80         // 一定会存在空位
81         if(h[hs] == null) return 0;
82         if(h[hs] == x) return 1;
83         hs++;
84         if(hs == N) hs = 0;
85     }
86     return 0;
87 }
88
89
90 int main() {
91     scanf("%d", &n);
92     // 初始化，一开始所有哈希值对应的数都设置为 null
93     // 0x3f3f3f3f 超出题目中的数据范围，所以可被视为 null
94     memset(h, 0x3f, sizeof(h));
95
96     char choose[5]; int x;
97     for(int i = 0; i < n; i++) {
98         scanf("%s", choose);
99         if(choose[0] == 'I') {
100             scanf("%d", &x);
101             insert(x);
102         } else {
103             scanf("%d", &x);
104             if(find(x)) printf("Yes\n");
105             else printf("No\n");
106         }
107     }
108
109     return 0;
110 }
111
```

字符串前缀哈希法

str = ABCABCD EYXCAcwing

① 预处理所有前缀的哈希

$$h[1] = \text{hash}(A) \quad h[2] = \text{hash}(AB) \quad h[3] = \text{hash}(ABC) \quad h[4] = \text{hash}(ABCA)$$

如何定义字符串的 hash?

把字符串看成是一个 p 进制的数, 每一位字母的 ASCII 码就是这个数的一位

例: ABCD 视为 4 位 **映射**

① A B C D 把 A 当成 1, B 当成 2, C 当成 3, D 当成 4

这样就是 $(1234)_p$ 对应 10 进制: $1 \times p^3 + 2 \times p^2 + 3 \times p + 4 \times p^0$

转成数字后, 再 $\% Q$ $(1 \times p^3 + 2 \times p^2 + 3 \times p + 4 \times p^0) \bmod Q \quad 0 \sim Q-1$

① 一般情况下, 不能映射成 0 若 A 映射为 0, 会有

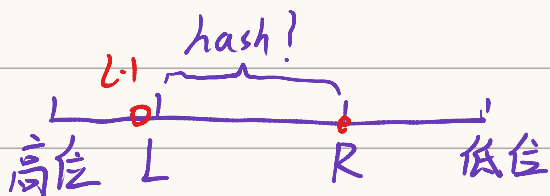
$A \rightarrow 0 \quad AA \rightarrow 0 \quad AAA \rightarrow 0$ 冲突了

② 这里是假定不存在冲突的, 完全不考虑冲突情况

经验值: $p = 131$ 或 $13331 \quad Q = 2^{64}$ 可以假定不会出现冲突

用 unsigned long long 存所用 h , 用溢出等价取模

② 利用前缀哈希求出所有子串的哈希值



已知 $h[R]$ R 为第 0 位, 1 为 $R-1$ 位

$h[L-1]$ $L-1$ 为第 0 位, 1 为 $L-2$ 位

$$h[R] \quad p^{R-1} \dots p^0$$

$$h[L-1] \quad p^{L-2} \dots p^0$$

L 到 R 段子串的
哈希值

① $L-1$ 左移 $h[L-1] \times p^{R-L+1}$

② 最终减去重叠段 $h[R] - h[L-1] \times p^{R-L+1}$

这个方法可用于判断两个字符串是否相同
哈希值相同视为字符串相同

841. 字符串哈希

题目 提交记录 讨论 题解 视频讲解

给定一个长度为 n 的字符串，再给定 m 个询问，每个询问包含四个整数 l_1, r_1, l_2, r_2 ，请你判断 $[l_1, r_1]$ 和 $[l_2, r_2]$ 这两个区间所包含的字符串子串是否完全相同。

字符串中只包含大小写英文字母和数字。

输入格式

第一行包含整数 n 和 m ，表示字符串长度和询问次数。

第二行包含一个长度为 n 的字符串，字符串中只包含大小写英文字母和数字。

接下来 m 行，每行包含四个整数 l_1, r_1, l_2, r_2 ，表示一次询问所涉及的两个区间。

注意，字符串的位置从 1 开始编号。

输出格式

对于每个询问输出一个结果，如果两个字符串子串完全相同则输出 **Yes**，否则输出 **No**。

每个结果占一行。

数据范围

$1 \leq n, m \leq 10^5$

输入样例：

```
8 3
aabbbaabb
1 3 5 7
1 3 6 8
1 2 1 2
```

输出样例：

```
Yes
No
Yes
```

难度：	简单
时/空限制：	1s / 64MB
总通过数：	81198
总尝试数：	125442
来源：	模板题
算法标签	

```
1 #include <stdio.h>
2
3 // 当我们要快速判断两个字符串是否相等的时候，就可以用这种做法
4 // 当我们将字符串哈希处理之后，就可以O1的时间判断是否相等了
5
6 typedef unsigned long long ULL;
7 // 用unsigned long long的溢出代替模2^64
8
9 const int N = 1e5 + 10;
10 const int P = 131;
11
12 int n, m;
13 char str[N];
14 ULL h[N], p[N];
15 // h[N] 用来存字符串前缀的哈希值，p[N]用来存储p的次方，这样就不用重复计算了
16
17 ULL getNum(int l, int r) {
18     // 这里需要用到p的k次，因此我们要提前处理出来
19     return h[r] - h[l - 1] * p[r - l + 1];
20 }
21
22 int main() {
23     scanf("%d%d%s", &n, &m, str + 1); // 字符串的编号从1开始
24
25     p[0] = 1; // p的0次方为1
26     for(int i = 1; i <= n; i++) { // 处理p和h
27         p[i] = p[i - 1] * P;
28         h[i] = h[i - 1] * P + str[i]; // 这里字母直接映射为ASCII码
29     }
30
31     for(int i = 0; i < m; i++) {
32         int l1, r1, l2, r2;
33         scanf("%d%d%d%d", &l1, &r1, &l2, &r2);
34         if(getNum(l1, r1) == getNum(l2, r2)) printf("Yes\n");
35         else printf("No\n");
36     }
37
38     return 0;
39 }
40
41
```

